TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO

HỌC PHẦN: NHẬP MÔN TỐI ƯU HÓA

NGÀNH: CÔNG NGHỆ THÔNG TIN

ĐỀ TÀI: TỐI ƯU SIÊU THAM SỐ RANDOM FOREST BẰNG RANDOM SEARCH

Sinh viên thực hiện: Phạm Phú Vĩnh
GVHD: Nguyễn Thị Diễm Anh

TP. Hồ Chí Minh – 10/2026

MỤC LỤC

TÓM TẮT 3
1. ĐẶT VẤN ĐỀ 4
Mục tiêu và đóng góp 4
2. MÔ HÌNH HÓA BÀI TOÁN 4
3. PHƯƠNG PHÁP 5
3.1. Baseline 5
3.2. Random Search 5
3.3. Công bằng và tái lập 5
4. DỮ LIỆU VÀ THIẾT KẾ THỰC NGHIỆM 5
5. KẾT QUẢ THỰC NGHIỆM 6
6. THẢO LUẬN VÀ PHÂN TÍCH 8
6.1. Phân tích độ nhạy: max_depth 8
6.2. Phân tích độ ổn định khi thay đổi seed 8
7. KẾT LUẬN VÀ ĐỀ XUẤT 9
7.1. Đường dẫn và hướng dẫn chạy 9
7.2. Công cụ hỗ trợ (AI Assistance) 9
PHỤ LỤC – MINH CHỨNG THỰC NGHIỆM 10
1. Mã nguồn thực nghiệm: 10
2. Nhật ký 30 lần thử: 10
3. Bảng đánh giá baseline và Random Search: 10
4. Kết quả bổ sung: 10
5. Hình được tạo từ kết quả chạy: 10
6. Sản phẩm báo cáo và bảo vệ: 10

TÓM TẮT

Nghiên cứu này đánh giá Random Search 30 lần thử để tối ưu bốn siêu tham số Random Forest trên bộ dữ liệu Breast Cancer Wisconsin. Đánh giá dùng 5-fold cross-validation trên tập huấn luyện và giữ riêng 20% dữ liệu để kiểm thử cuối. Baseline đạt CV macro-F1 0.9603, phương pháp tìm kiếm đạt 0.9576; hai phương pháp đạt cùng test macro-F1 0.9526. Do vậy, trong phạm vi thực nghiệm, cấu hình mặc định được ưu tiên do không phát sinh chi phí tìm kiếm.

1. ĐẶT VẤN ĐỀ

Random Forest là một phương pháp học có giám sát thường dùng cho bài toán phân loại. Các lựa chọn như số lượng cây, độ sâu và số mẫu tối thiểu tại nút có thể ảnh hưởng đến chất lượng dự đoán, độ phức tạp và chi phí chạy. Cấu hình mặc định là một baseline hợp lý nhưng không phải lúc nào cũng tối ưu.

Câu hỏi nghiên cứu: Trong budget tối đa 30 cấu hình, Random Search có cải thiện macro-F1 so với Random Forest mặc định không? Nếu không, cấu hình nào được chọn khi xét thêm chi phí tính toán?

Phạm vi: một bộ dữ liệu benchmark, một họ mô hình, Random Search và baseline. Dự án tập trung tối ưu hóa và thực nghiệm thay vì phát triển giao diện.

Mục tiêu và đóng góp

Mô hình hóa biến quyết định/hàm mục tiêu/giới hạn; triển khai baseline và search theo cùng protocol; lưu lại mọi trial; phân tích hội tụ, chất lượng nghiệm, thời gian, độ nhạy, độ ổn định; đưa ra quyết định có căn cứ số liệu.

2. MÔ HÌNH HÓA BÀI TOÁN

Biến quyết định: x = (n_estimators, max_depth, min_samples_split, min_samples_leaf). Tập nghiệm Ω là tích Descartes các tập rời rạc trong bảng sau.

Thành phần	**Miền giá trị**
n_estimators	20, 40, 60, 80, 100
max_depth	None, 3, 5, 10, 15, 20, 30
min_samples_split	2, 4, 6, 10, 15, 20
min_samples_leaf	1, 2, 3, 5, 7, 10

Objective: max f(x) = (1/5) × Σ[k=1..5] MacroF1_k(x), tính trên 5 fold validation được xác định từ tập training. Mỗi cấu hình là một nghiệm hợp lệ trong Ω.

Ràng buộc: x ∈ Ω; số lượng cấu hình Random Search không vượt quá 30 trials; dùng cùng train/test split, folds, metric và fixed seed. Test set không tham gia tối ưu. Chi phí thời gian được đo và dùng như yếu tố trade-off hậu kiểm, không nằm trong hàm mục tiêu chính.

Điều kiện dừng: hết 30 trials. Feasibility được đảm bảo bởi việc lấy mẫu các cấu hình trong miền giá trị hợp lệ.

3. PHƯƠNG PHÁP

3.1. Baseline

RandomForestClassifier với giá trị mặc định của scikit-learn (trừ random_state=42 và n_jobs=1 để chạy xác định và kiểm soát CPU). Baseline được đo bằng cùng 5-fold stratified CV trên training split.

3.2. Random Search

Sử dụng ParameterSampler lấy 30 cấu hình khác nhau trong miền Ω bằng seed 42. Với từng cấu hình, huấn luyện/đánh giá bằng cùng 5 folds, ghi mean/std CV macro-F1, runtime và best-so-far. Nghiệm có mean CV macro-F1 cao nhất được chọn để huấn luyện lại trên toàn bộ training split.

Pseudocode: chia dữ liệu → đo baseline CV → lặp 30 trials: lấy x, tính CV macro-F1(x), cập nhật best-so-far, ghi CSV → fit baseline/best x trên train → đánh giá mỗi mô hình trên test → phân tích.

3.3. Công bằng và tái lập

Các phương pháp dùng cùng dataset, split 80/20, folds stratified và macro-F1. Baseline là một cấu hình tham chiếu và Random Search được cấp 30 evaluations; báo cáo nêu rõ chi phí bổ sung. Hệ thống thư viện, seed và tham số được lưu trong experiment_config.json.

4. DỮ LIỆU VÀ THIẾT KẾ THỰC NGHIỆM

Dataset: Breast Cancer Wisconsin (Diagnostic), bộ dữ liệu công khai truy cập qua sklearn.datasets.load_breast_cancer. Gồm 569 mẫu, 30 đặc trưng số và 2 lớp. Nguồn tài liệu: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html.

Preprocessing: sử dụng các đặc trưng số trong dataset; Random Forest không yêu cầu chuẩn hóa trên bộ dữ liệu này; không thực hiện chọn thuộc tính dựa trên test set. Train/test split phân tầng tỷ lệ 80/20, random_state=42. Trên training split, StratifiedKFold(n_splits=5, shuffle=True, random_state=42).

Metric chính: macro-F1 trung bình qua 5 folds; metric phụ: holdout test macro-F1, accuracy, runtime và độ lệch chuẩn CV. Bộ test chỉ được đọc vào để đánh giá sau khi chọn best configuration bằng CV.

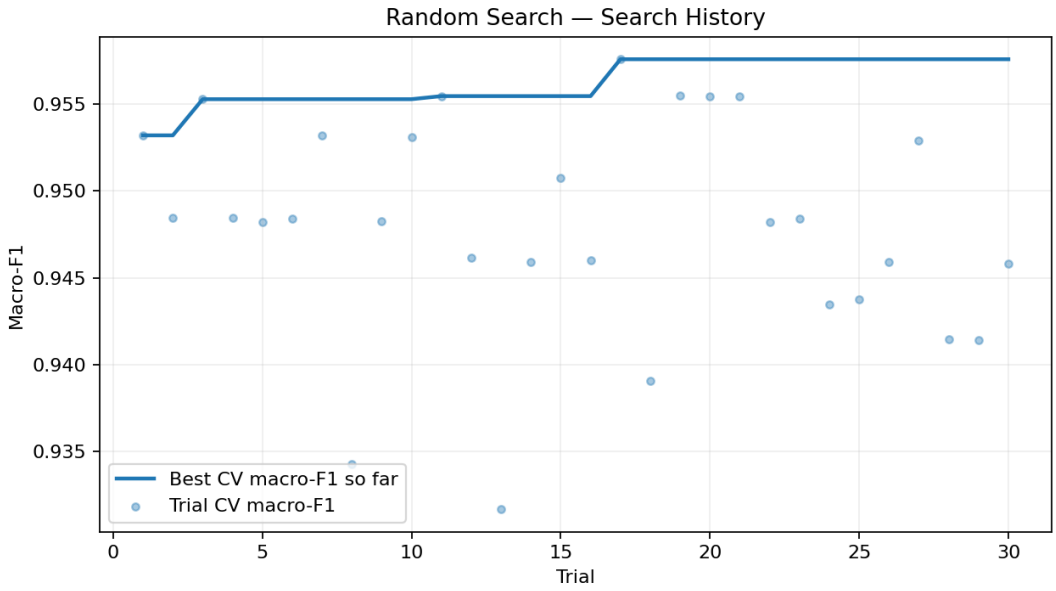
Thực nghiệm bổ sung: sensitivity thay đổi max_depth quanh cấu hình được chọn; stability thay đổi random_state của Random Forest với các giá trị 11, 23, 42 trong khi giữ nguyên split/folds. Các phép kiểm tra này không tương đương chạy lại toàn bộ Random Search với nhiều seed.

Môi trường ghi trong experiment_config.json của lần chạy mới: Python 3.13.16, scikit-learn 1.6.1. Runtime phụ thuộc môi trường và phần cứng; không so sánh trực tiếp với lần chạy cũ.

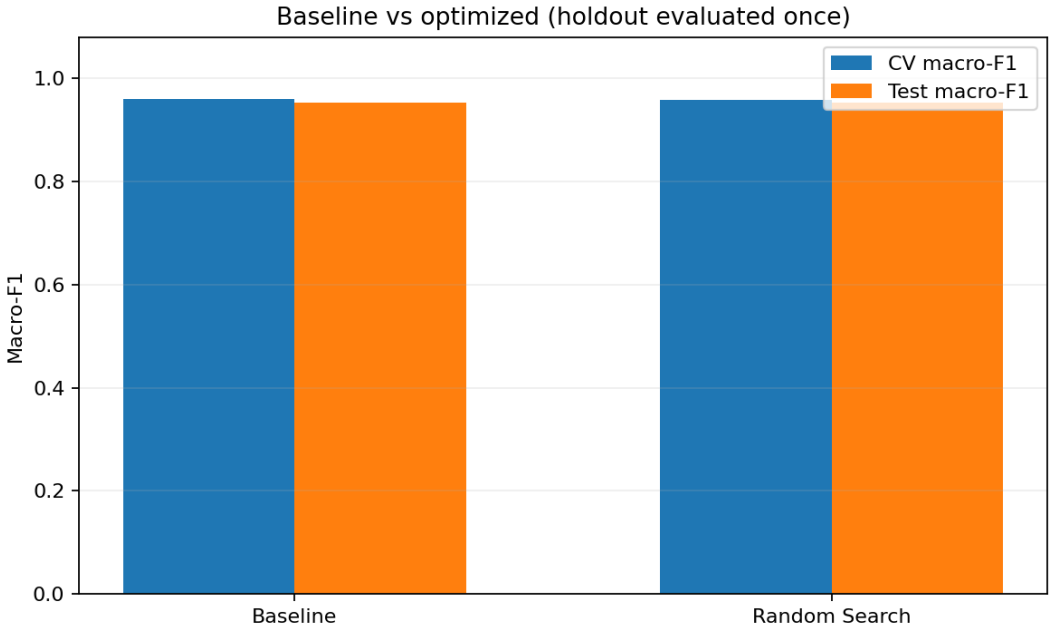
5. KẾT QUẢ THỰC NGHIỆM

Chỉ số	**Default RF**	**Random Search**
CV Macro-F1	0.9603	0.9576
CV std	0.0188	0.0190
Test Macro-F1	0.9526	0.9526
Test Accuracy	0.9561	0.9561
CV evaluation cost	0.98s	14.96s
Final fit time	0.171s	0.135s

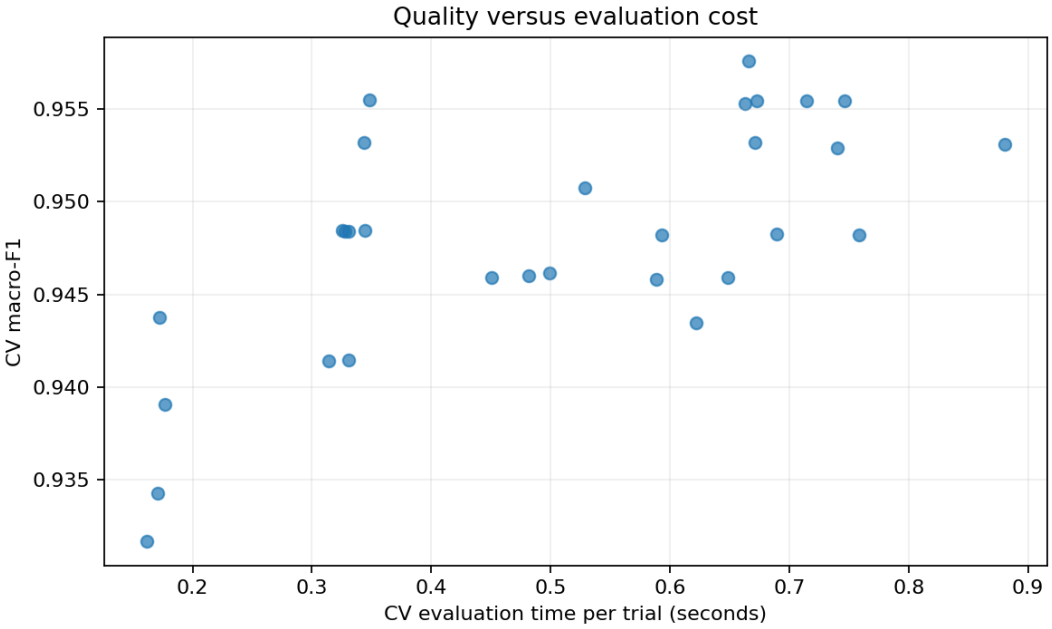
Cấu hình có điểm CV tốt nhất trong Random Search: {'n_estimators': 80, 'min_samples_split': 4, 'min_samples_leaf': 2, 'max_depth': 5}. Lưu ý kết quả test không dùng trong việc lựa chọn.



Hình 1. Hội tụ Random Search từ 30 trial (nguồn: results/trial_history.csv).



Hình 2. So sánh CV Macro-F1 và Test Macro-F1 từ results/comparison.csv.



Hình 3. Trade-off giữa CV Macro-F1 và thời gian đánh giá cấu hình.

6. THẢO LUẬN VÀ PHÂN TÍCH

Kết quả cho thấy baseline CV macro-F1 (0.9603) nhỉnh hơn cấu hình tốt nhất được Random Search tìm thấy (0.9576); chênh lệch 0.0027. Trên test set, cả hai đều đạt macro-F1 0.9526; như vậy chưa có chứng cứ về cải thiện hiệu suất khái quát hóa trong thực nghiệm này.

Random Search cần 14.96 giây để đánh giá 30 cấu hình trên môi trường chạy Colab (theo comparison.csv), chưa tính thí nghiệm bổ sung. Baseline cần 0.98 giây cho một lần đánh giá CV. Đây là search cost; không đánh đồng với fit runtime của một mô hình cuối cùng.

6.1. Phân tích độ nhạy: max_depth

max_depth	**CV macro-F1**
3.0	0.9507
5.0	0.9576
10.0	0.9555
20.0	0.9555
nan	0.9555

6.2. Phân tích độ ổn định khi thay đổi seed

Phương pháp	**Mean macro-F1**	**Std across 3 seeds**
Baseline	0.9594	0.0014
Optimized	0.9553	0.0024

Hạn chế: Một bộ dữ liệu nhỏ, chỉ một random-search seed, một phân hoạch train/test, miền tìm kiếm chưa bao trùm mọi tổ hợp và thời gian chạy lệ thuộc phần cứng. Stability ở đây phản ánh estimator seed, không kiểm chứng độ biến thiên của search qua nhiều lần khởi chạy. Không diễn giải kết quả thành chỉ dẫn sử dụng trong y khoa.

7. KẾT LUẬN VÀ ĐỀ XUẤT

Theo protocol đã công bố, Random Search không vượt baseline về CV macro-F1 và cũng không tạo ra lợi ích trên held-out test macro-F1. Vì vậy, quyết định cuối là giữ Random Forest mặc định cho trường hợp này: đơn giản, khả năng tái lập tốt, không phát sinh chi phí tìm kiếm 30 trials.

Trong tương lai có thể tăng budget, dùng Optuna/TPE với cùng budget để kiểm chứng thêm; chạy lại nhiều seed tìm kiếm và nhiều split; thử thêm dataset khác; hoặc xây dựng multi-objective optimization cho chất lượng và latency. Các hướng này chưa được tuyên bố là đã thực hiện trong bài hiện tại.

7.1. Đường dẫn và hướng dẫn chạy

GitHub repository: https://github.com/PhuVinh206/random-forest-hyperparameter-optimization

Lệnh chạy: pip install -r requirements.txt; python src/run_experiments.py (chạy từ gốc thư mục dự án).

Nguồn: tài liệu đề bài môn Nhập môn Tối ưu hóa; Hướng dẫn phạm vi đề; scikit-learn Breast Cancer dataset documentation.

7.2. Công cụ hỗ trợ (AI Assistance)

Có sử dụng AI hỗ trợ dựng mã nguồn, cấu trúc repo, soạn thảo và trình bày báo cáo. Dữ liệu số trong bảng và hình được sinh từ lần thực thi mã đính kèm. Sinh viên cần tự chạy lại, kiểm chứng và giải thích được mọi lựa chọn trước khi nộp.

PHỤ LỤC – MINH CHỨNG THỰC NGHIỆM

Minh chứng có sẵn trong bộ ZIP RandomForest_HPO_DA_CHAY(1).zip và repository GitHub:

1. Mã nguồn thực nghiệm: src/run_experiments.py; tái lập môi trường: requirements.txt và results/experiment_config.json (Python 3.13.16, scikit-learn 1.6.1, seed 42, 5 folds, 30 trials).

2. Nhật ký 30 lần thử: results/trial_history.csv; file lựa chọn nghiệm: results/best_config.json; nhật ký thực thi: results/run_log.txt.

3. Bảng đánh giá baseline và Random Search: results/comparison.csv; CV Macro-F1 lần lượt 0.9603 và 0.9576; Test Macro-F1 đều 0.9526.

4. Kết quả bổ sung: results/sensitivity.csv, results/stability.csv, results/confusion_matrix.csv và results/classification_report.txt.

5. Hình được tạo từ kết quả chạy: figures/convergence.png, figures/comparison.png và figures/tradeoff.png; đã đưa vào Chương 5.

6. Sản phẩm báo cáo và bảo vệ: thư mục report/, slides/ và README.md; GitHub: https://github.com/PhuVinh206/random-forest-hyperparameter-optimization.

Chú ý về tiến độ 5 tuần: các tệp này xác nhận sản phẩm kỹ thuật và kết quả đã sinh ra sau khi chạy trên Colab; không thể thay thế minh chứng lịch sử commit theo từng tuần. Người học cần dùng lịch sử commit thực tế trên GitHub để chứng minh tiến độ.